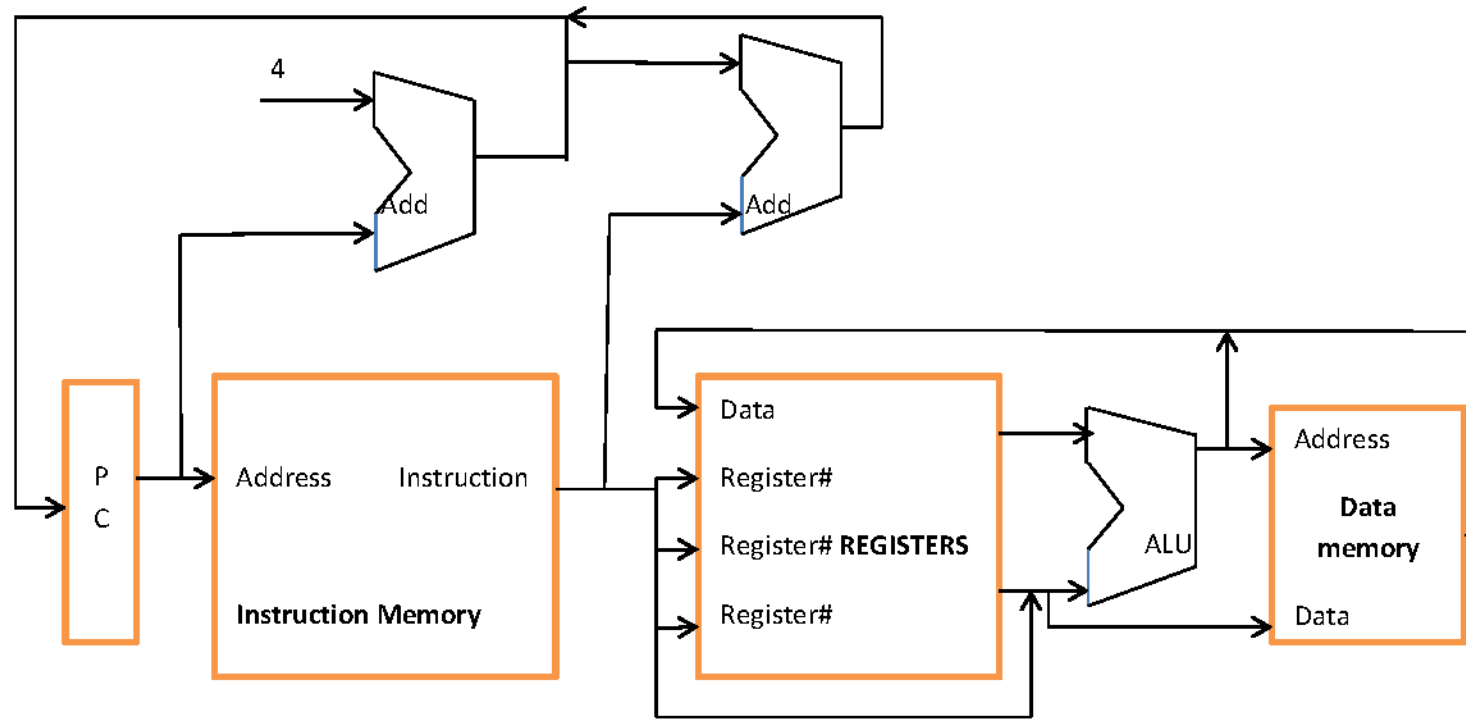


The Processor:



- *Abstract view of the implementation of MIPS subset showing the major functional units and the major connections between them.*
- a) All the instructions start by using PC to supply the instruction address to instruction memory.
- b) After the instruction is fetched, the register operands used by an instruction are specified by the fields of that instruction.
- c) Once the register operands have been fetched, they can be operated on to compute the memory address (for load or store), to compute an arithmetic result (for an integer arithmetic logical instruction) or compare (for branch).

d)If the instruction is arithmetic logical instruction, the result from ALU must be written to a register.

e)If the operation is load or store,the ALU result is used as an address to either store a value from the registers or load a value from memory into registers. The result from the ALU or memory is written back into the register file.

f) Branches require the use of ALU output to determine the next instruction address, which comes from either the ALU (where the PC and the branch offset are summed) or from an adder that increments the current PC by 4.

g) All the instructions (memory reference, arithmetic-logical and branch) except jump use ALU after reading the registers.

Logic design conventions

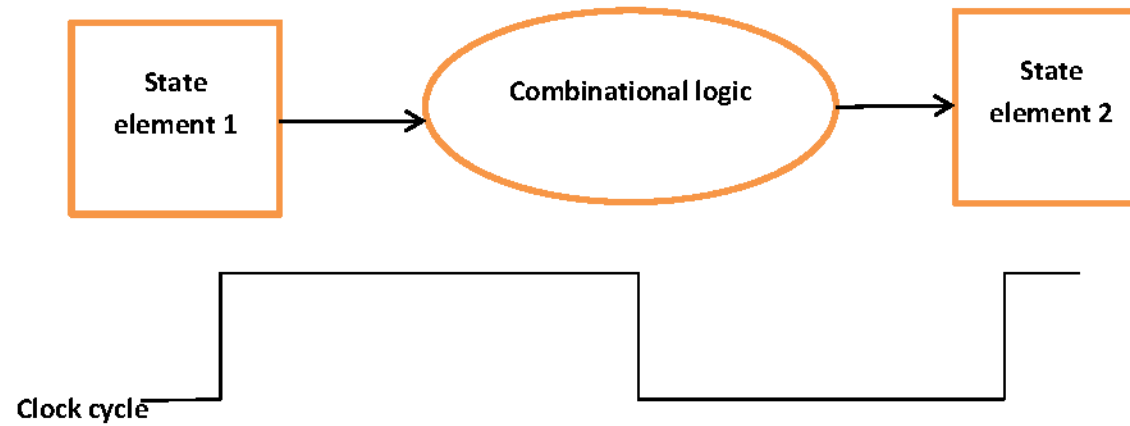
- The functional units in MIPS implementation consists of two different types of logic elements: elements that operate on data values and elements that contain state.
- Elements that operate on data values are called combinational. Which means that their output depends only on the current inputs. Given the same inputs a combinational element always produces the same output.(Ex ALU).
- An element contain state if it has some internal storage :state elements, because if we pulled the plug on the m/c,we could restart it by loading the state elements with the values they contained before we pulled the plug.
- The instruction and data memories ,registers are state elements.A state element has two inputs and one output.

- Inputs are the data value to be written into the element and the clock, which determines when the data value is written. Output from the state element provides the value that was written in an earlier clock cycle.

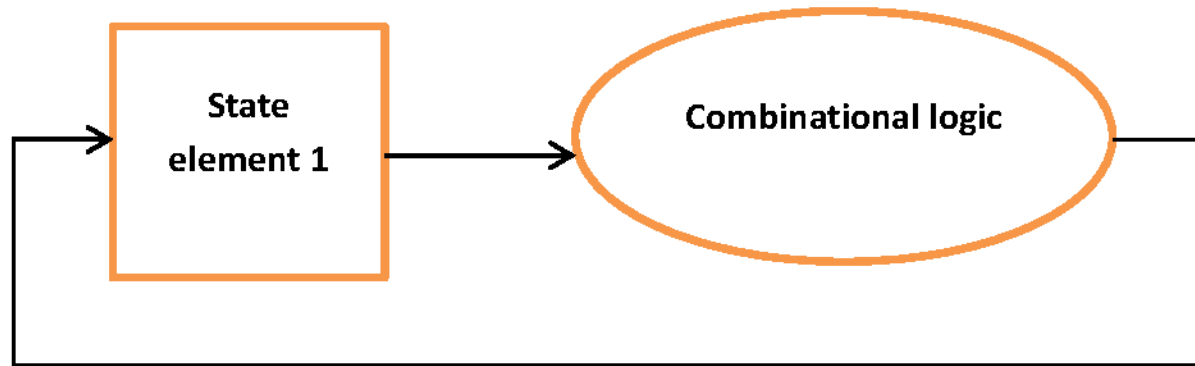
Ex D-type flip-flop.

- Logic components that contain state: sequential (output depends on both inputs and contents of internal state). Ex registers.
- *Clocking Methodology*: when the data is valid and stable relative to the clock.
- *Edge-triggered clocking*: A clocking scheme in which all state changes occur on a clock edge. It means any values stored in a sequential logic element are updated only on a clock edge.
- The inputs are the values that were written in previous clock cycle, while the outputs are values that can be used in a following clock cycle.

Combinational Logic, state element and the clock are closely related

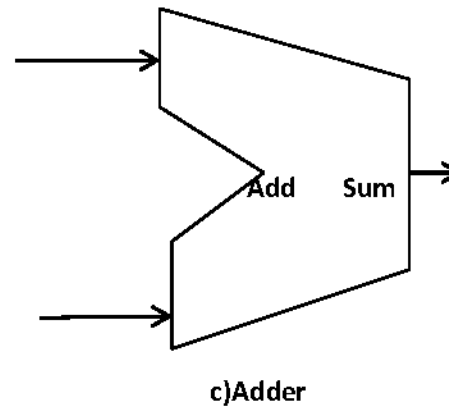
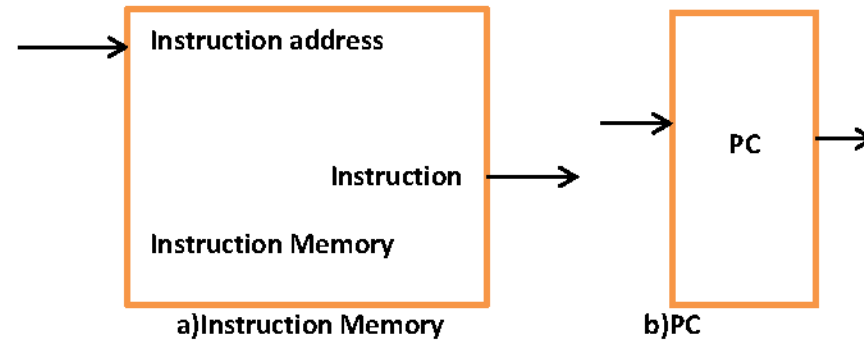


Edge triggered methodology allows a state element to be read and written in the same clock cycle without creating a race that could lead to indeterminate data values

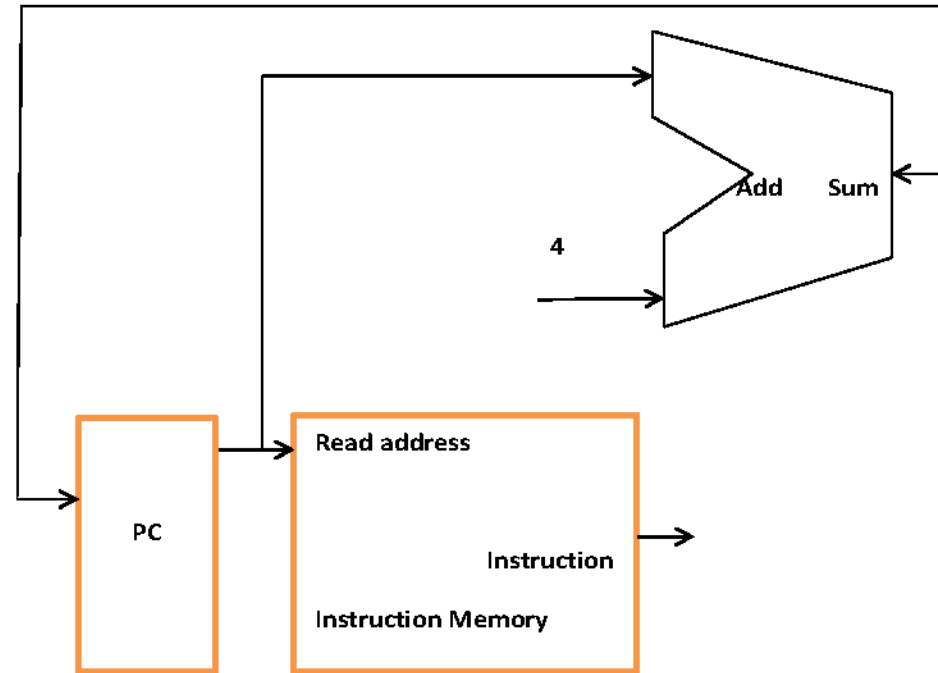


- Control signal: A signal used for multiplexor selection or for directing the operation of a functional unit.
- Contrasts with a data signal ,which contains information that is operated on by a functional unit.

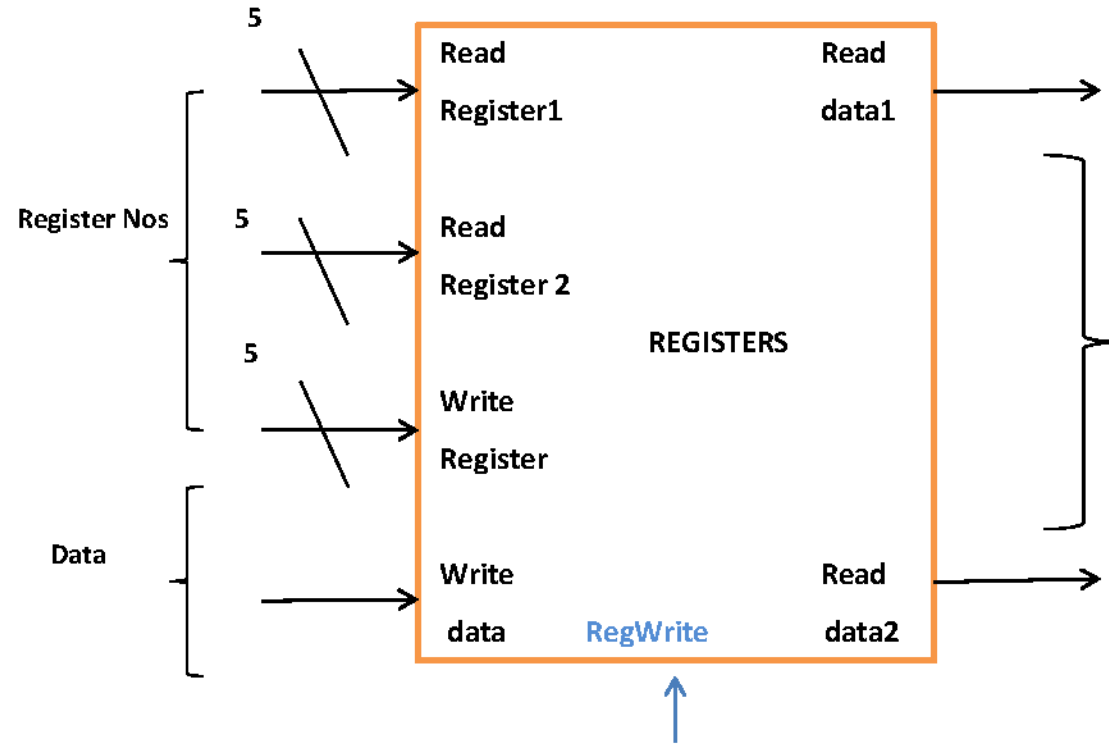
Building a datapath



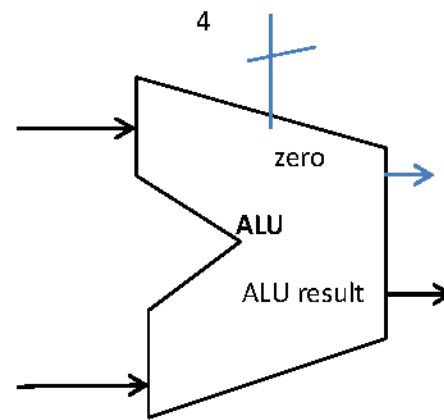
Portion of datapath used for fetching instructions and incrementing the PC

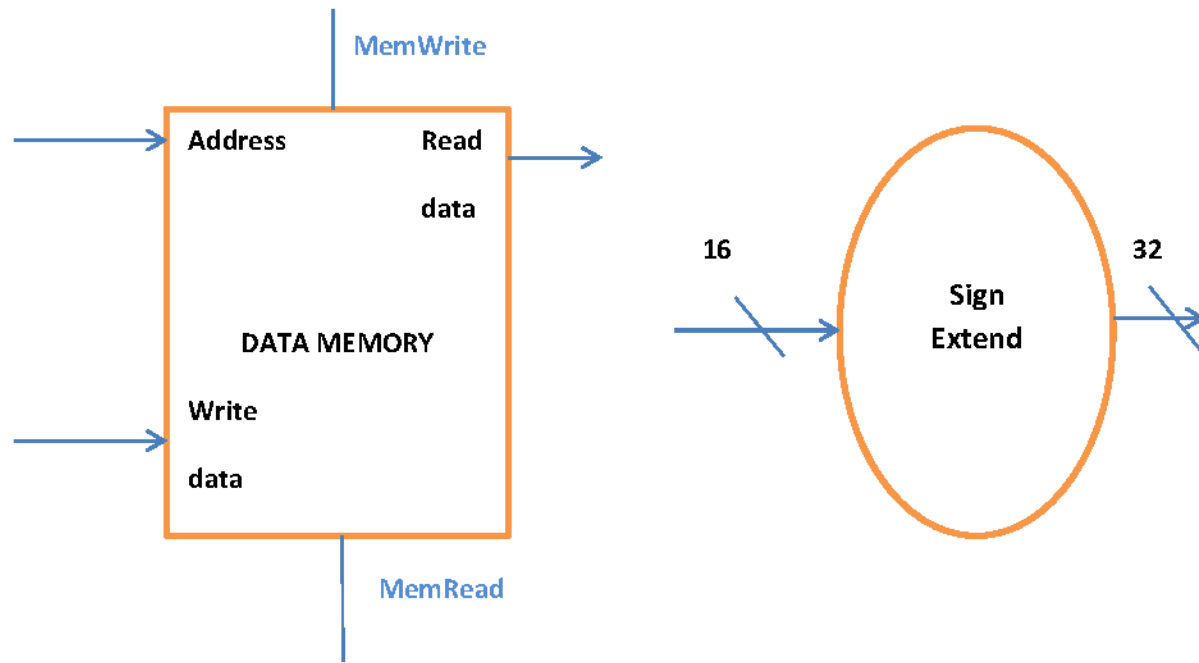


Register file

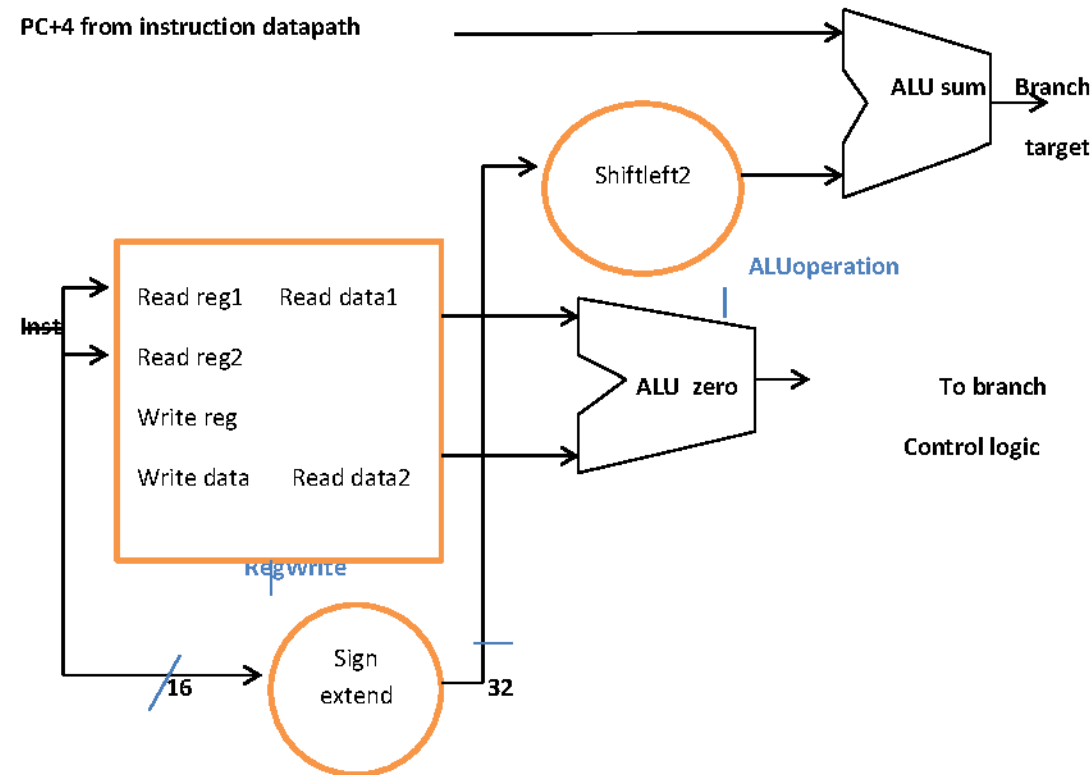


ALU Operation





Datapath for branch uses the ALU to evaluate the branch condition and a separate adder to compute the branch target as the some of the incremented PC and the sign-extended, lower 16 bits of the instruction shifted left two bits.



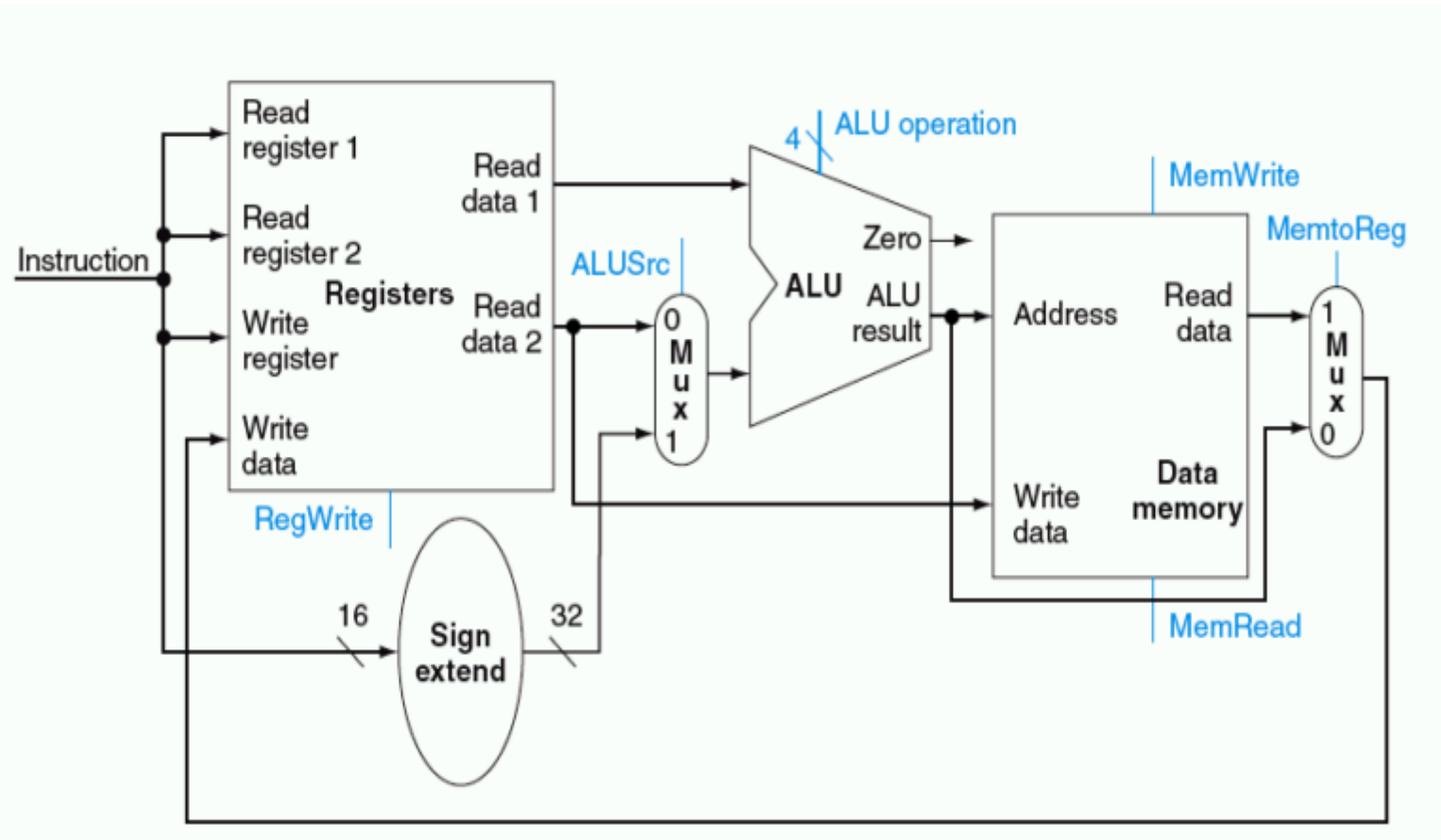
Creating single datapath

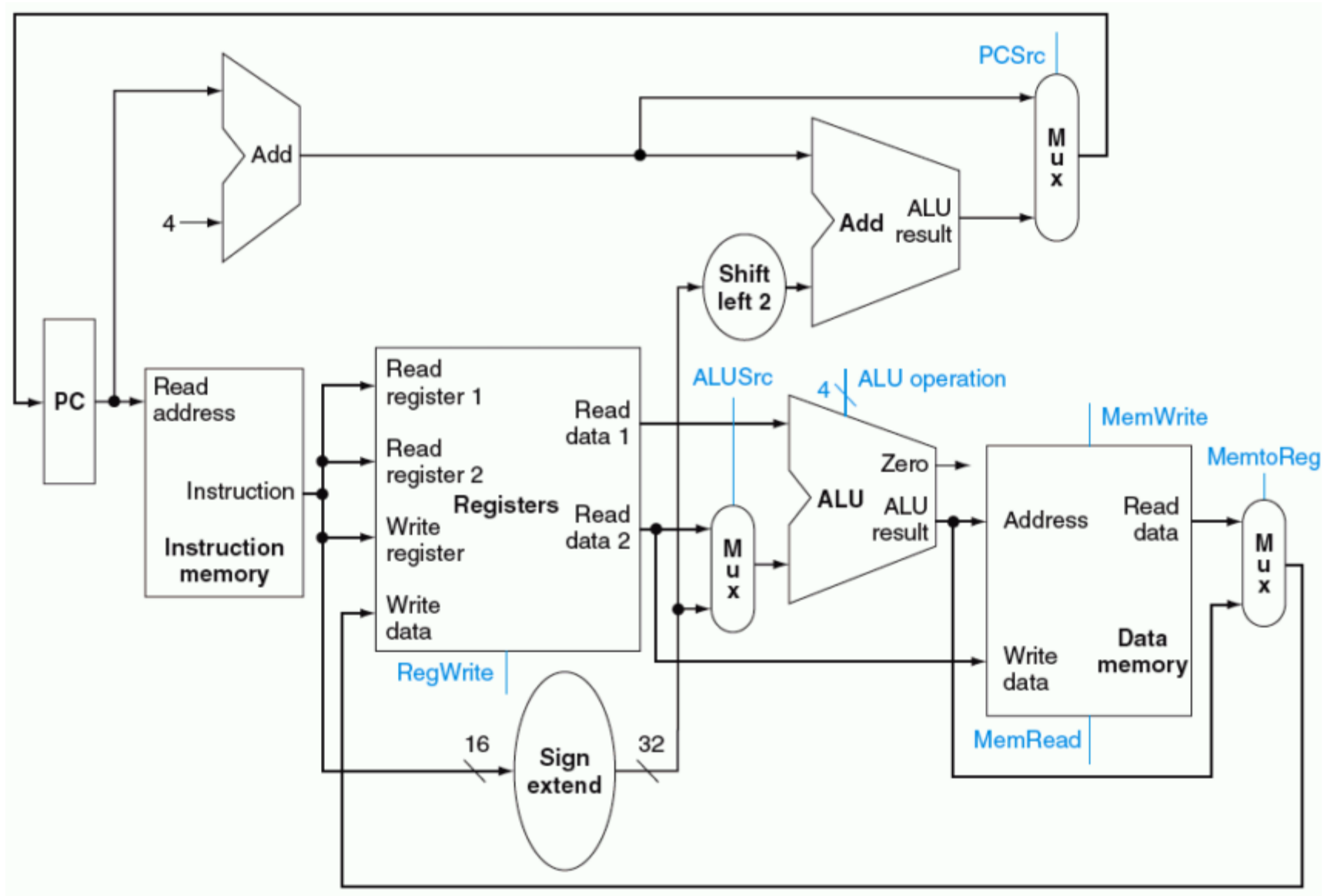
1)To share a datapath element between two different instruction classes,we may need to allow multiple connections to the input of the element using a multiplexor and control signal to select among the multiple inputs.

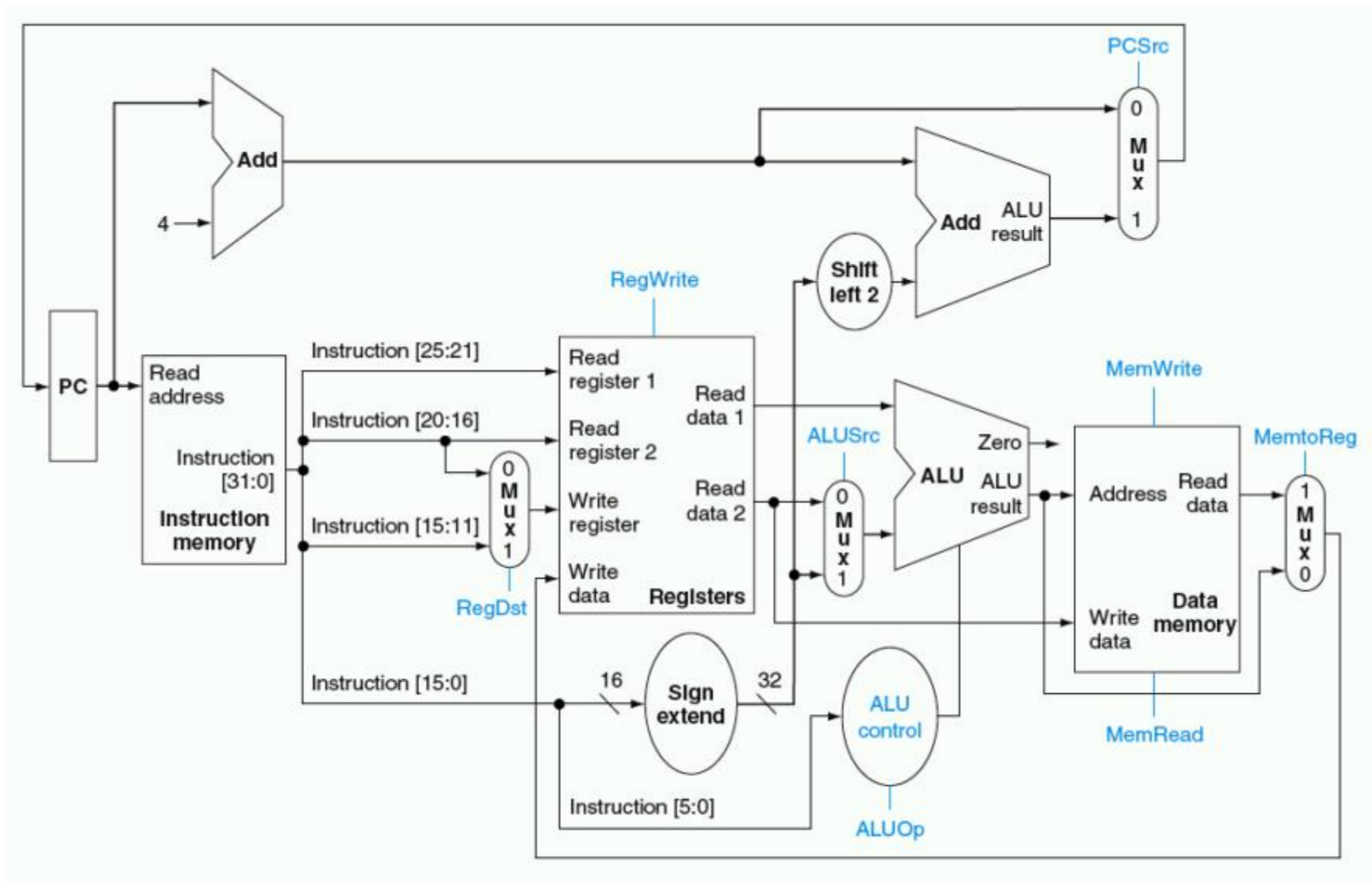
2)The operations of R-type and memory instructions datapath are quite similar but the differences are:

a)The R-types uses ALU with inputs coming from two registers.The memory instruction also use ALU for address calculation.Although the second input is the sign-extended 16 bit offset field from instruction.

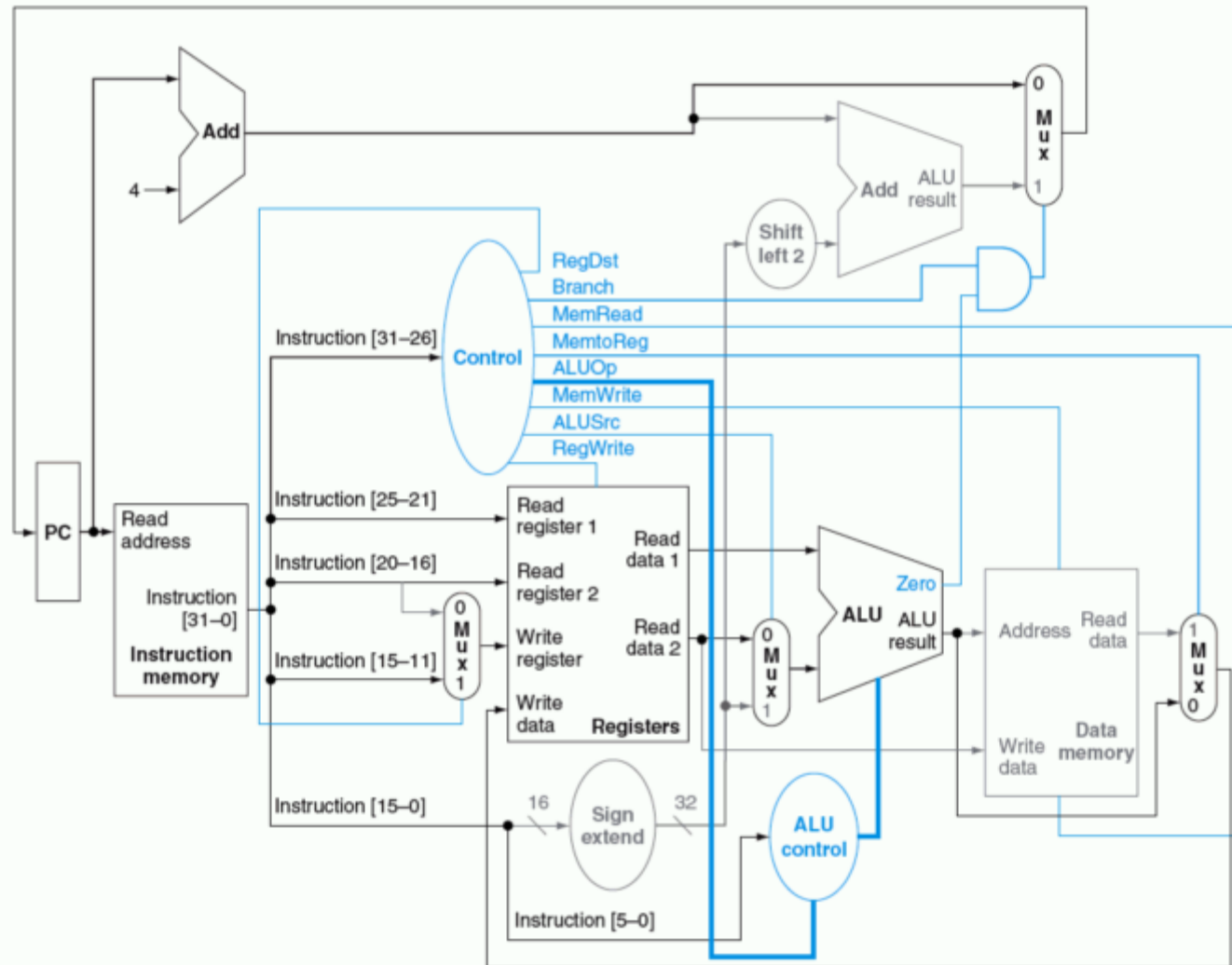
b)The value stored into the destination register comes from ALU(R-type) or the memory(for load)







add \$t1,\$t2,\$t3



Steps:

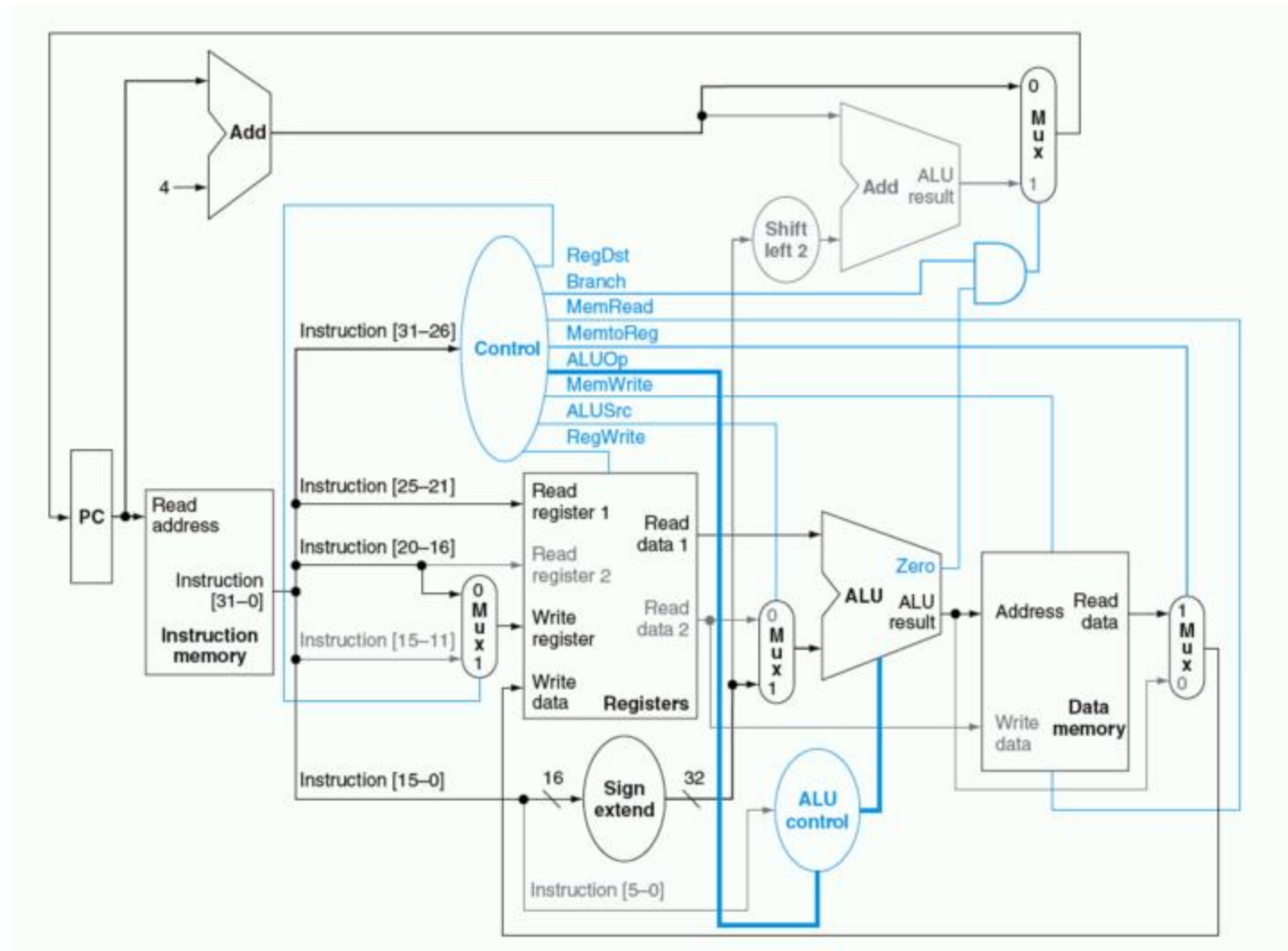
1)The instruction is fetched and the PC is incremented

2)Two registers,\$t2 and \$t3 are read from the register file ;also,
the main control unit computes the setting of the control lines during this step.

3)The ALU operates on the data read from the register file ,using the
function code(bits 5:0,which is the function field, of the instruction)to generate
The ALU function.

4)The result from the ALU is written into the register file using bits 15:11 .
of the instruction to select the destination register \$t1.

lw \$t1,offset(\$t2)



Steps:

1)The instruction is fetched from the instruction memory and the PC is incremented.

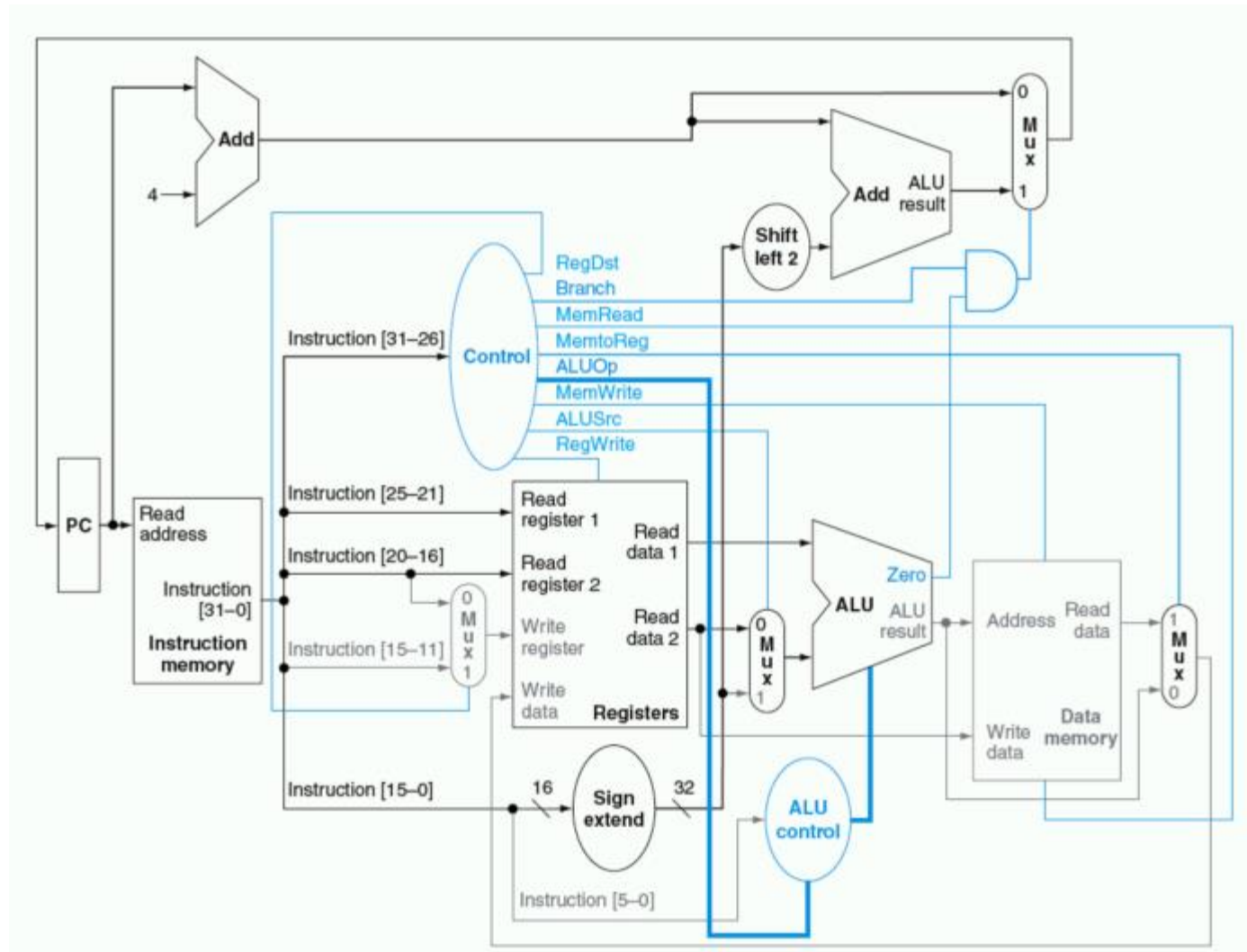
2)A register (\$t2)value is read from the register file.

3)The ALU computes the sum of the value read from the register file and the signed extended, lower 16 bits of the instruction(offset)

4)The sum from the ALU is used as the address for the data memory.

5)The data from the memory unit is written into the register file; the register destination is given by bits 20:16 of the instruction(\$t1).

Beq \$t1,\$t2,offset



Steps:

1)The instruction is fetched from the instruction memory and the PC is incremented.

2)Two register \$t1 and \$t2 are read from the register file.

3)The ALU performs a subtract on the data values read from the register. The Value of PC+4 is added to sign extended, lower 16 bits of the instruction(offset) Shifted left by two;the result is the branch target address.

4)The zero result from ALU is used to decide which adder result to store into the PC

5)The data from the memory unit is written into the register file; the register destination is given by bits 20:16 of the instruction(\$t1).

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register destination number for the Write register comes from the rt field (bits 20:16).	The register destination number for the Write register comes from the rd field (bits 15:11).
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of $PC + 4$.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.